Daffodil International University
Department of Software Engineering

System Design Document

Unit 2: Order, Payment & Escrow

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 14 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Unit 1: Identity & Access (`01_identity_access.pdf`)
Database schema	`docs/design/sql/02_orders_payments.sql`, tested on PostgreSQL (45 checks and a concurrency test, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 Unit Overview **4**
2.1 Responsibilities . . . 4
2.2 Components . . . 4
2.3 New convention: transactional ports . . . 5

3 Interfaces **5**
3.1 HTTP API . . . 5
3.2 Provider adapters . . . 6
3.3 Ports provided to other units . . . 7
3.4 Events published . . . 7
3.5 Ports required . . . 8

4 Data Design **8**
4.1 Tables . . . 8
4.2 The guarantees, and why they live in the database . . . 9
4.3 The ledger . . . 11
4.4 The split . . . 11

5 Behaviour **12**
5.1 Order and escrow states . . . 12
5.2 Deadlines . . . 12
5.3 Fixed-price checkout . . . 13
5.4 Auction and second-chance orders . . . 13
5.5 Payment . . . 13
5.6 Fulfilment steps and the delivery event . . . 15
5.7 Release . . . 15
5.8 Cancellation and deadline refunds . . . 15
5.9 Refunds . . . 16
5.10 Disputes . . . 16
5.11 Payout accounts and payouts . . . 16
5.12 Reconciliation and chargebacks . . . 17
5.13 Monetization . . . 17
5.14 Scheduled jobs . . . 17

6 Security Design **18**
6.1 Verifying provider messages . . . 18
6.2 Card and wallet data . . . 18
6.3 Credentials, environments and idempotency . . . 18
6.4 Abuse and fraud controls . . . 19
6.5 Operators . . . 19

7 Parameters **19**

8 Testing **19**

9 Notes for Integration **20**

A Traceability **20**

B Database Schema and Tests **22**
B.1 Test results . . . 22
B.2 Schema . . . 23

1 Introduction

1.1 Purpose

This document is the second unit of TrustBazaar's System Design Document. It specifies how the *Order, Payment & Escrow* unit meets its requirements in SRS v2.0. Every purchase in TrustBazaar, whether at a fixed price, by auction or for a game account, passes through this unit: it creates the order, collects the payment, holds it in escrow, and releases or refunds it according to the rules. It also keeps the ledger that shows where every poisha went. Acceptance criteria 2, 3, 4 and 6 of the SRS (Appendix A) depend on this unit.

The unit follows the conventions DC-1 to DC-12 of Unit 1 (`01_identity_access.pdf`, Section 2), adds one convention (DC-13, Section 2.3), and uses Unit 1's ports for identity checks, step-up authentication, approvals and audit.

1.2 Scope

In scope:

- orders for fixed-price purchases, auction wins and second-chance offers, with checkout holds and payment windows;
- the choice of fulfilment method and the fixed delivery fee;
- payment collection through bKash, Nagad and Stripe, with confirmation, deduplication and reconciliation;
- the escrow state machine and every way money leaves it: release, auto-release, the game-account protection hold, cancellation, deadline refunds and dispute outcomes;
- refunds; seller payout accounts and payouts; the double-entry ledger and settlement records;
- commission and delivery margin; featured-listing fees and Pro Seller subscriptions; chargebacks.

Out of scope, owned by later units:

- listing stock and its display (Listings; this unit reserves stock through a port);
- the auction close, second-chance selection and non-payment strikes (Auction; this unit reports expired payment windows);
- courier booking, handover codes and fares (Delivery; this unit receives fulfilment steps and the delivery event);
- credential handover and takeover (Game-Account; this unit receives credential delivery and takeover completion);
- the dispute process itself (Disputes; this unit freezes escrow and applies outcomes);
- the delivery of notifications (Notifications; this unit emits events).

1.3 Requirements covered

- **Functional:** FR-PAY-01 to FR-PAY-21; FR-MON-01 to FR-MON-10. On the order side: FR-FIX-01 to FR-FIX-05, FR-AUC-14 and FR-AUC-15. On the escrow side: FR-DLV-11 to FR-DLV-13, FR-GAM-10, FR-GAM-17, FR-DSP-02, FR-DSP-06, FR-DSP-10 and FR-DSP-11. Also FR-TNS-05 (payout holds), FR-TNS-07 (order limits) and FR-ACC-11 (closure check).
- **Security:** SEC-PAY-01 to SEC-PAY-07; SEC-LOG-01 for money events.
- **Other:** IF-SW-01 to IF-SW-03; IF-UI-04, IF-UI-05, IF-UI-08; NFR-PERF-06, NFR-PERF-08; NFR-REL-02 to NFR-REL-04; NFR-CMP-02, NFR-CMP-03, NFR-CMP-05; DR-03, DR-05, DR-10.

2 Unit Overview

2.1 Responsibilities

1. Create an order only when stock is reserved and the buyer may transact, and fix its price, fees and fee rates at that moment.
2. Collect payment on the provider's own page, accept it only on a verified server-side confirmation, and accept each payment at most once.
3. Hold the money in escrow and let it leave only by the transitions of SRS Table 4.8, under the deadlines of the order.
4. Record every movement in a balanced double-entry ledger, from which the settlement record and the revenue log are derived.
5. Pay sellers only into verified accounts, after the cooling-off period, unless a fraud hold applies.
6. Detect and correct anything the providers and the ledger disagree on.

2.2 Components

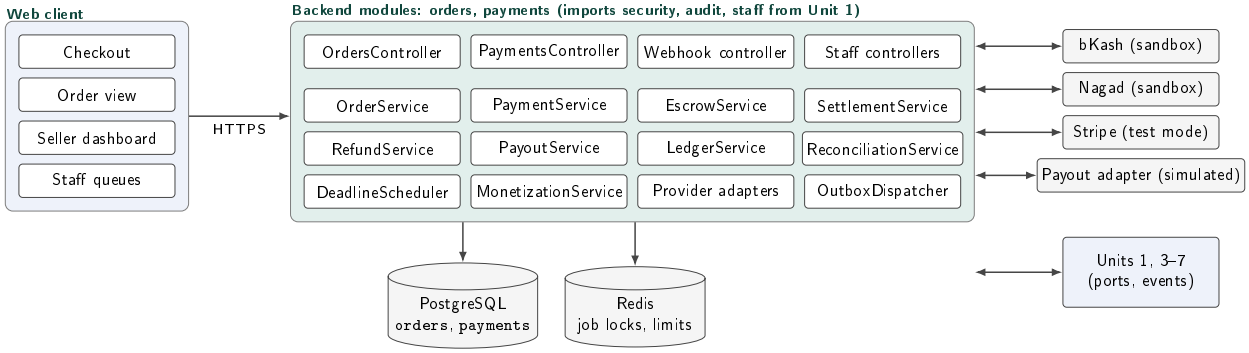


Figure 1: Components of the Order, Payment & Escrow unit. Each provider is reached only through its adapter; provider pages are opened in the user's browser by redirect.

Component	Responsibility
OrderService	Creates orders (fixed-price checkout; auction and second-chance orders through the port); sets fulfilment and the delivery fee; enforces order limits; cancels unpaid orders.
PaymentService	Starts payment attempts; confirms them from verified provider evidence; handles duplicate, late and mismatched payments.
Webhook controller (ProviderEventsController)	Receives Stripe webhooks and bKash and Nagad browser callbacks; verifies them (signature, or verification query) and records each event once.
EscrowService	The only component that changes escrow state; applies fulfilment steps, delivery events, buyer releases, cancellations, deadline refunds and dispute outcomes.
SettlementService	Computes the split, writes the settlement record and posts the ledger entries on every terminal transition.
LedgerService	Posts balanced journal entries; provides balances and the revenue log.
RefundService, PayoutService	Execute refunds and payouts through adapters, with idempotency keys, retries and manual fallback.
ReconciliationService	Compares in-flight attempts, refunds and payouts with the providers, and raises discrepancies for operators.
DeadlineScheduler	Runs the scheduled actions of Section 5.14.
MonetizationService	Featured-listing purchases and Pro Seller subscriptions; answers "is this seller Pro?".

Component	Responsibility
Provider adapters	One per provider, behind one interface (Section 3.2); a simulated payout adapter.

2.3 New convention: transactional ports

DC-13. A port may take the caller's transaction (`EntityManager`) as an argument when two units must change state atomically. The callee still writes only its own tables. This unit uses the rule in exactly three places:

- reserving stock while creating an order (`ListingsPort.reserve`, FR-FIX-02);
- recording a fulfilment step or delivery event reported by Delivery or Game-Account;
- applying a dispute outcome decided in Disputes.

Every other interaction between units goes through events. To avoid deadlocks, every transaction that locks more than one row locks them in a fixed order: listing, then order, then escrow, then attempt.

3 Interfaces

3.1 HTTP API

All paths are under `/api/v1`; policies and rate limits are those of Unit 1 (Section 4.1). Amounts in requests are never trusted (SEC-PAY-06): every price, fee and total is computed on the server from the listing, the delivery quote and the order.

Method	Path	Policy	Purpose (requirement)
Buyer			
`POST`	`/orders`	Verified(buy)	Fixed-price checkout: reserve stock, create order (FR-FIX-01, 02, 05)
`GET`	`/orders?role=buyer|seller`	SignedIn	My orders
`GET`	`/orders/{id}`	Party to order	Order, escrow state, deadlines (IF-UI-05)
`PUT`	`/orders/{id}/fulfilment`	Buyer	Choose method and delivery quote (FR-DLV-02, 09)
`POST`	`/orders/{id}/payments`	Verified(buy), buyer	Start payment; returns provider redirect (FR-PAY-01)
`GET`	`/payments/return/{provider}`	Public	Landing page after the provider; shows status only, never confirms (FR-PAY-02)
`POST`	`/orders/{id}/confirm-receipt`	Buyer	Release after the delivery event (FR-PAY-07a)
`POST`	`/orders/{id}/release-early`	Buyer, StepUp	Game account: release before the protection period ends (FR-GAM-17)
`POST`	`/orders/{id}/cancellation-requests`	Buyer	Ask to cancel before the delivery event (FR-PAY-09)
`GET`	`/orders/{id}/receipt`	Party to order	Receipt or statement (FR-PAY-21)
Seller			
`POST`	`/orders/{id}/cancellation-requests/{rid}/decision`	Seller	Accept or decline (FR-PAY-09)
`POST`	`/orders/{id}/cancel`	Seller	Cancel after payment, before delivery; refund and strike (FR-PAY-09)
`GET`	`/me/payout-account`	Verified	Current account and status

Method	Path	Policy	Purpose (requirement)
POST	/me/payout-account	Verified(payout), StepUp	Add or replace (FR-PAY-17, 18)
GET	/me/payouts, /me/statements	Verified	Payout history, settlement statements
POST	/listings/{id}/feature	Verified(list), seller	Buy featured placement (FR-MON-05)
POST, DELETE	/me/subscription	Verified	Start or cancel Pro Seller (FR-MON-06, 08)
Providers (no session; verified per Section 6.1)			
POST	/webhooks/stripe	Stripe signature	Checkout, refund, dispute events
GET	/callbacks/bkash/{attemptId}	Signed state	bKash browser callback; triggers Execute and Query
GET	/callbacks/nagad/{attemptId}	Signed state	Nagad browser callback; triggers Verify
Staff			
GET	/staff/payments/issues	Staff(operator)	Reconciliation queue (FR-OPS-01)
POST	/staff/payments/issues/{id}/resolve	Staff(operator)	Resolve with a recorded reason
GET	/staff/payout-accounts/pending	Staff(operator)	Bank accounts awaiting the name check
POST	/staff/payout-accounts/{id}/decision	Staff(operator)	Approve or reject (FR-PAY-17)
POST	/staff/payouts/{id}/release-hold	Staff(operator)	Lift a fraud hold (FR-TNS-05)

3.2 Provider adapters

Each provider is wrapped in an adapter implementing one interface, so the rest of the unit never sees a provider's own API. The exact endpoints, field names and signing schemes follow each provider's documentation [SRS R13] and are confined to the adapter.

```
interface PaymentProvider {
  readonly id: 'bkash' | 'nagad' | 'stripe';
  createPayment(a: { attemptId: string; amount: bigint; idempotencyKey: string;
      returnUrl: string }): Promise<{ providerPaymentId: string; redirectUrl: string }>;
  // turns provider evidence into a verified result; never trusts the browser
  verify(input: CallbackInput | WebhookInput): Promise<VerifiedResult>;
  query(attempt: Attempt): Promise<VerifiedResult>;        // reconciliation and expiry checks
  refund(r: { attempt: Attempt; amount: bigint; idempotencyKey: string }):
      Promise<{ status: 'succeeded' | 'pending' | 'unsupported'; providerRefundId?: string }>;
}
type VerifiedResult = { eventId: string; providerPaymentId: string; providerTxnId?: string;
  status: 'succeeded' | 'failed' | 'pending'; amount: bigint; currency: string;
  merchantReference: string /* = attemptId */; verifiedBy: 'signature' | 'query' };
interface PayoutProvider { send(p: { payoutId: string; amount: bigint; destination: PayoutAccount;
  idempotencyKey: string }): Promise<{ status: 'sent' | 'failed'; ref?: string }>; }
```

Provider	How a payment is made and confirmed
bKash	Tokenized Checkout. The adapter obtains a grant token and creates a payment, passing the attempt id as the merchant invoice number. The buyer enters wallet number, OTP and PIN on bKash's page. bKash redirects the browser to our callback, carrying the payment id and a status. *The callback status is not trusted.* The server calls Execute Payment, and Query Payment if the outcome is unclear, and accepts only a completed transaction whose amount, currency and invoice number match. Refunds use bKash's refund function with the payment id, transaction id and amount; partial refunds are supported.

Provider	How a payment is made and confirmed
Nagad	Merchant checkout. The adapter initialises and completes a checkout with the attempt id as the order id; the buyer pays on Nagad's page. The callback carries a payment reference, which the server verifies with Nagad's verification API before accepting. If the sandbox offers no refund function, a refund is marked `manual_required` and placed in the operator queue (FR-PAY-12).
Stripe	Stripe Checkout, the hosted payment page, reached by redirect, in test mode. The attempt id is sent as the client reference, and the attempt's idempotency key on every API call. Confirmation comes from the `checkout.session.completed` webhook, verified with the `Stripe-Signature` header within the CP-32 tolerance. Refunds use the refunds API with an idempotency key. `charge.dispute.created` records a chargeback (FR-PAY-20). Because the card form is entirely on Stripe's page and is reached by redirect, the merchant is eligible for SAQ A without the script-protection criterion that applies to embedded forms [SRS R6].
Payouts	No disbursement sandbox is assumed (SRS OI-4). The simulated adapter records a reference and marks payouts `simulated`; a real adapter replaces it without other changes.

3.3 Ports provided to other units

```
interface OrdersPort {                                  // used by the Auction unit
  createAuctionOrder(a: { listingId: string; winnerId: string; price: bigint;
      kind: 'auction' | 'second_chance' }): Promise<Order>;   // payment window CP-10 / CP-11
  getOrderForParty(orderId: string, userId: string): Promise<OrderView>;
}
interface EscrowCommands {                              // used by Delivery, Game-Account, Disputes
  recordFulfilmentStep(orderId: string, step: FulfilmentStep, at: Date, tx: EntityManager): Promise<void>;
  recordDeliveryEvent(orderId: string, source: DeliverySource, at: Date, tx: EntityManager): Promise<void>;
  refundCourierReturned(orderId: string, courierCharge: bigint, tx: EntityManager): Promise<void>;
  freezeForDispute(orderId: string, disputeId: string, tx: EntityManager): Promise<void>;
  applyDisputeOutcome(orderId: string, disputeId: string, o: DisputeOutcome,
      tx: EntityManager): Promise<void>;                // release | full | partial(itemRefund)
  withdrawDispute(orderId: string, disputeId: string, tx: EntityManager): Promise<void>;
}
interface EscrowQuery {
  getEscrow(orderId: string): Promise<EscrowView>;          // state, deadlines, amounts
  disputeAllowed(orderId: string, at: Date): Promise<boolean>;   // held and inside the window
}
interface ProStatusQuery { isPro(userId: string): Promise<boolean> }   // Listings: limit CP-28
// Unit 1's AccountClosureCheck is implemented here: open orders, unpaid wins, balance owed.
```

3.4 Events published

All are written to `orders.outbox` in the same transaction as the change (DC-6).

Event	Consumers and purpose
`orders.order.created`	Notifications.
`orders.order.paid`	Listings: mark units sold (FR-FIX-04). Delivery: book the courier or open the meetup flow. Game-Account: ask the seller for credentials (FR-GAM-10). Notifications.
`orders.order.cancelled`	Listings: return reserved stock (FR-FIX-03). Notifications.
`orders.order.payment_expired`	Auction: second-chance offer and non-payment strike (FR-AUC-15, FR-AUC-16).
`orders.order.delivered`	Notifications: dispute window and release date (FR-NTF-07).
`orders.escrow.frozen`	Notifications.
`orders.escrow.released`, `.refunded`, `.partially_refunded`	Reviews: rating unlocked (FR-REV-01). Notifications.
`orders.order.cancelled_by_seller`	Trust & Safety: seller strike (FR-PAY-09, FR-TNS-04).

Event	Consumers and purpose
`payments.refund.manual_required`, `payments.payout.failed`	Notifications; operator queue.
`payments.payout_account.changed`	Notifications: alert to the seller's number and e-mail (FR-PAY-18).
`payments.featured.started`, `.ended`; `payments.subscription.changed`	Listings: featured label, listing limit (FR-MON-05, FR-MON-08).

3.5 Ports required

Port	Provider and use
`IdentityQuery (Unit 1)`	`assertCanTransact`; `getVerifiedPhone` for MFS payout accounts; `getKycLegalName` for the bank-account check; the date of verification for CP-27 (an addition to the Unit 1 port, noted in Section 9).
`StepUpGuard`, `ApprovalService`, `AuditService (Unit 1)`	Step-up for payout changes and early release; audit of every money action.
`ListingsPort (Unit 3)`	`reserve(listingId, qty, tx)` and `release(listingId, qty, tx)`: one conditional update each (DC-13); `snapshot(listingId)` for the order.
`DeliveryPort (Unit 4)`	`getQuote(quoteId)`: fee, expiry, buyer and listing it was issued for.
`GameAccountPort (Unit 5)`	`assertCheckoutReady(orderId)`: disclosure acknowledged and buyer public key present (FR-GAM-05, FR-GAM-07).
`ConfigPort`	CP values; their version is stored on each order.

4 Data Design

The schema is in `docs/design/sql/02_orders_payments.sql` (Appendix B). It is loaded after Unit 1's schema and references only `identity.users` and `identity.staff_accounts` (DC-5). The unit owns two schemas: `orders` (orders, escrow and its history, cancellation requests, outbox) and `payments` (attempts, provider events, refunds, payout accounts, payouts, ledger, settlements, reconciliation issues and monetization).

4.1 Tables

Table	Holds	Rules enforced by the database
`orders.orders`	One row per order with a snapshot of the listing and the fee rates in force (DR-03, FR-MON-10).	Buyer is not the seller; item price = unit price × quantity; total = item price + fee; auctions have quantity 1; a fee only for courier, and courier only with a quote; game accounts only by game-account handover; no payment before a fulfilment method; a paid order has a fulfilment deadline; a delivery event only after the fulfilment step; at most one live order per auction listing.

Table	Holds	Rules enforced by the database
`orders.escrows`	One row per paid order.	Starts in `held`; only the transitions of SRS Table 4.8 (trigger); amounts consistent with the state (nothing paid out while open; released + refunded = collected when terminal; a partial refund has both); a frozen escrow names its dispute; the dispute window ends no later than auto-release; terminal rows never change; the collected amount never changes.
`orders.escrow_history`	Every transition with actor, reason and amounts (FR-PAY-10).	Append-only (trigger and grants).
`orders.cancellation_requests`	Buyer requests before the delivery event.	One pending request per order.
`payments.attempts`	Each attempt to pay, with its idempotency key and provider references.	BDT only; **one successful attempt per order** (acceptance criterion 4); provider references unique.
`payments.provider_events`	Every verified webhook, callback or query result.	**Unique per provider and event id**, so a replay has no effect (SEC-PAY-04).
`payments.refunds`	Each refund, linked to the original payment.	The total refunded from a payment can never exceed it (trigger).
`payments.payout_accounts`	Wallet or bank details (account number encrypted).	One current account per seller; a bank account is usable only after an operator's name check; a usable account has a cooling-off end time.
`payments.payouts`	One payout per order.	One per order; an account is required unless the payout is waiting for one or held; a hold records its reason.
`payments.journal_entries / _lines`	The double-entry ledger.	Every entry balances (deferred constraint trigger, checked at commit); append-only.
`payments.settlements`	The settlement record of FR-PAY-13.	Parts add up exactly to the amount collected; courier payout + margin $\leq$ delivery fee; append-only.
`payments.ledger_accounts`	The chart of accounts.	Not changeable by the application role.

4.2 The guarantees, and why they live in the database

1. **An order is paid only once** (AC-4). The confirmation path is idempotent in code, and the database adds two independent barriers. A provider event can be recorded only once, and an order can have only one `succeeded` attempt. A second payment that really succeeded at the provider, for example from two browser tabs, is stored as `succeeded_duplicate` and refunded in full. The concurrency test of Appendix B.1 sends 15 confirmations at once, 10 copies of one webhook and 5 different confirmations; exactly one escrow results.
2. **Money leaves escrow only by permitted transitions** (AC-3, AC-6). The trigger `escrow_transition` rejects any other change and any change to a terminal escrow, whatever the application does.
3. **Every poisha is accounted for** (FR-PAY-14). Every journal entry must sum to zero when its transaction commits, and every settlement record must add up to the amount collected. A refund can never exceed its payment.
4. **History cannot be rewritten.** Escrow history, journal entries and lines, and settlements are append-only by trigger, and the application role has no `UPDATE` on them.



Figure 2: Orders and escrow (schema `orders`). Each paid order has exactly one escrow row, whose state can change only along the edges of Figure 4.

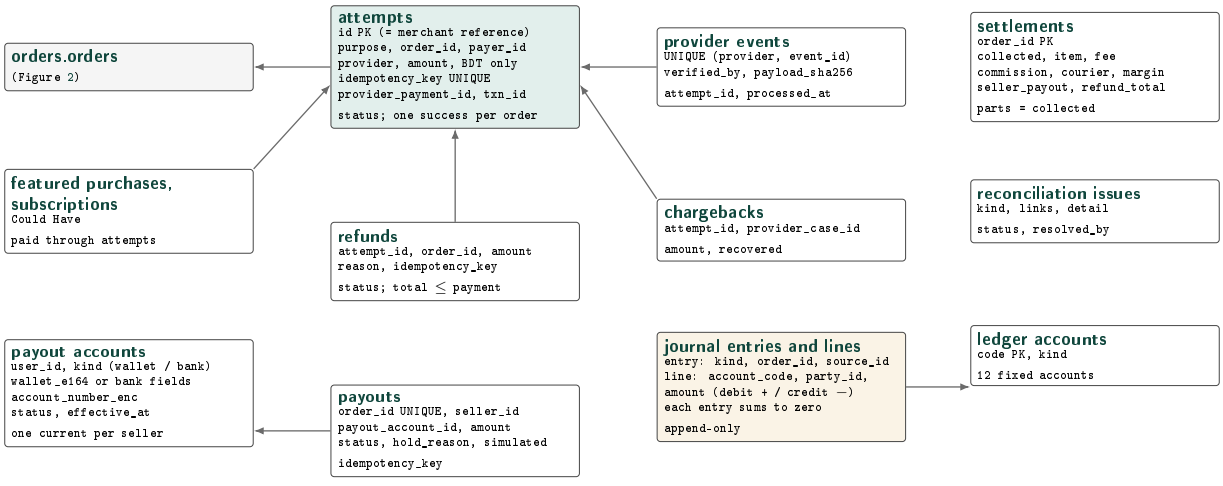


Figure 3: Payments, payouts and the ledger (schema `payments`). Settlements, refunds, payouts and journal entries also reference `orders.orders`; those arrows are omitted for legibility.

4.3 The ledger

Debits are positive and credits negative, so each entry sums to zero. `party_id` identifies the order, seller or buyer a line belongs to. Table 7 lists the entries the unit posts; every money event in the system is one of them.

Table 7: Journal entries posted by the unit (Dr = debit, Cr = credit).

Event	Entry
Payment confirmed	Dr provider_clearing (total) / Cr escrow_liability [order] (total).
Release	Dr escrow_liability (total) / Cr seller_payable [seller], revenue_commission, courier_payable, revenue_delivery_margin.
Full refund	Dr escrow_liability (total) / Cr buyer_refund_payable [buyer].
Partial refund	Dr escrow_liability (total) / Cr buyer_refund_payable, seller_payable, revenue_commission, and courier_payable with revenue_delivery_margin if the courier picked up.
Refund paid out	Dr buyer_refund_payable / Cr provider_clearing.
Duplicate, late or mismatched payment	Dr provider_clearing / Cr buyer_refund_payable (money in, owed straight back).
Payout sent	Dr seller_payable / Cr payout_clearing.
Courier settled	Dr courier_payable / Cr payout_clearing.
Courier return charge	Dr expense_courier_returns / Cr courier_payable (FR-DLV-12).
Featured fee, subscription	Dr provider_clearing / Cr revenue_featured or revenue_subscription.
Chargeback	Dr seller_receivable [seller] / Cr provider_clearing; recovered later by Dr seller_payable / Cr seller_receivable.

The settlement record of an order is derived from the same numbers as its release or refund entry and is written in the same transaction. The revenue log (FR-MON-09) is the view `payments.revenue_log`: every line on a `revenue_*` account. It is separate from the settlement records, as the SRS requires. In production the `provider_clearing` and `escrow_liability` balances correspond to funds held by the Bangladesh Bank-licensed provider (SRS C-7); the platform instructs movements but does not hold the money itself.

4.4 The split

`payments.split(item_price, delivery_fee, item_refund, courier_done, commission_bps, margin_bps)` is the reference implementation of SRS FR-PAY-11, FR-PAY-14, FR-MON-01 and FR-DLV-13. The application's `SettlementService` implements the same rules, and a test compares the two on random inputs.

1. A refund is taken from the item price first: *released* = item price − item refund.
2. Commission = $\lfloor (released \times commission_bps + 5000)/10000 \rfloor$, which is half-up rounding to the poisha.
3. If the courier picked the item up, the delivery fee is earned. The margin is computed in the same way, and the courier payout = fee − margin. Otherwise the whole fee is refunded.
4. Seller payout = *released* − commission, calculated last as the remainder.

The parts therefore always sum to item price + fee. The test script checks this on the SRS worked example (BDT 500 + 15 + 135 + 9,500 = 10,150) and on its partial-refund variant. It also checks 20,000 random cases, which all add up exactly with no negative part.

An early draft of the function took the refund as a single amount and could leave part of an unused delivery fee unallocated. Restating it in terms of the *item* refund, with the fee rule applied

separately, removed that case.

5 Behaviour

5.1 Order and escrow states

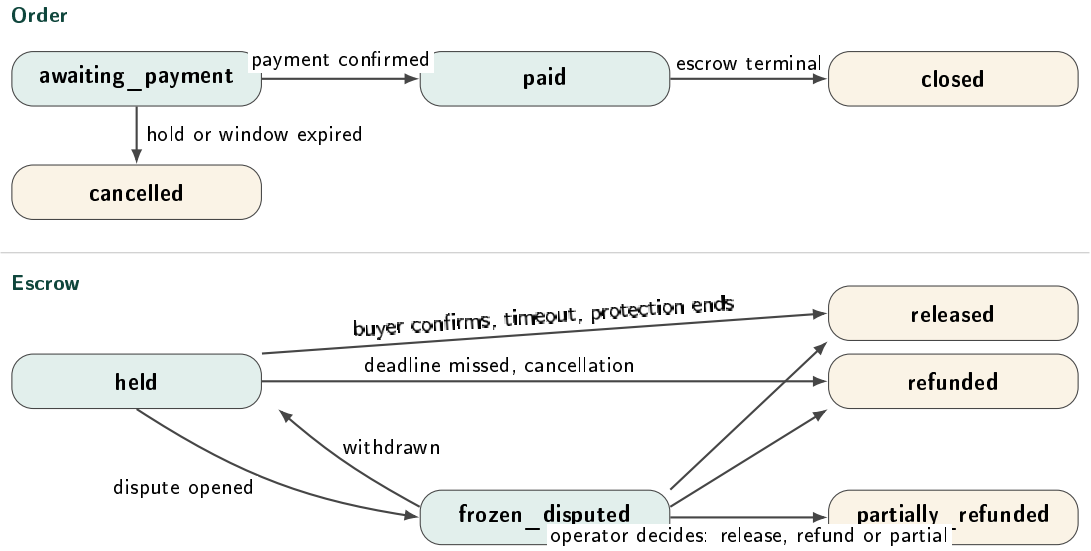


Figure 4: Order status (top) and escrow state (bottom). Terminal states are shaded. The escrow edges are exactly those of SRS Table 4.8, and the database trigger refuses any other.

5.2 Deadlines

Every order carries the deadlines that apply to it as columns, so a scheduled job can find due work with an index scan and act on recorded times (NFR-REL-03). Figure 5 shows them in order.

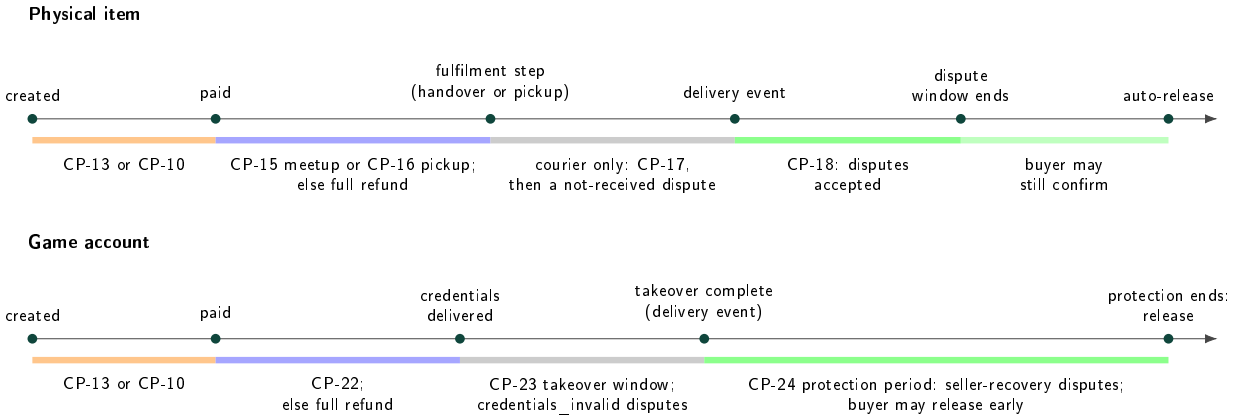


Figure 5: Deadlines of an order. Each segment's rule is enforced by a scheduled job (Section 5.14) from the times stored on the order and its escrow.

- `payment_due_at`: the checkout hold `CP-13` for fixed-price orders, or the payment window `CP-10` (or `CP-11` for a second-chance order) for auctions.
- `fulfilment_due_at`, set at payment: `CP-15` for a meetup handover, `CP-16` for courier pickup, `CP-22` for credential delivery. The step to be met is stored in `fulfilment_step`.
- At the delivery event:

- standard orders: `dispute_window_ends_at` = event + `CP-18` and `auto_release_at` = event + `CP-19`;
- game-account orders (`game_protection`): both = event + `CP-24`, because disputes are accepted throughout the protection period (FR-GAM-16, FR-GAM-17).

All values come from the parameter version stored on the order, so a later parameter change never alters a running order (FR-OPS-08).

5.3 Fixed-price checkout

`POST /orders` with listing, quantity, fulfilment method and, for courier, a delivery quote id. One transaction (DC-13):

1. `@Verified(buy)` has already passed. Check the order limits CP-27 for buyers verified fewer than 30 days ago (FR-TNS-07), and refuse if the buyer is the seller.
2. `ListingsPort.reserve(listing, qty, tx)` runs one conditional update in the Listings schema: decrease available and increase reserved, *only if* the listing is active and available $\geq$ qty. Zero rows updated means "not available", returned at once as `409`. Simultaneous checkouts therefore get definite answers and never over-reserve (FR-FIX-02, FR-FIX-05). Because the update is a single statement, PostgreSQL's row lock serialises concurrent requests without a lost update.
3. For courier, `DeliveryPort.getQuote` must return an unexpired quote issued to this buyer for this listing; its fee becomes `delivery_fee`. For a game account, the fulfilment is `game_account` and the fee is zero.
4. Insert the order with the listing snapshot, the fee rates (the commission rate depends on `ProStatusQuery`) and the parameter version, with `payment_due_at` = now + CP-13. Write `orders.order.created` to the outbox and audit.

5.4 Auction and second-chance orders

When an auction closes, the Auction unit calls `OrdersPort.createAuctionOrder` with the winner and price; the payment window is CP-10. The buyer then chooses the fulfilment method (`PUT /orders/{id}/fulfilment`) and pays.

If the window passes unpaid, the scheduled job cancels the order (`payment_window_expired`) and emits `orders.order.payment_expired`. The Auction unit then gives the strike and makes the second-chance offer. An accepted offer becomes a `second_chance` order with window CP-11. The partial unique index allows only one live order per auction listing, so a second-chance order can exist only after the first is cancelled.

5.5 Payment

- **Start.** Payment can start only while the order is `awaiting_payment` with a fulfilment method. The game-account unit must report the order ready: the disclosure acknowledged and the buyer's public key present. The attempt's amount is the order total. Starting a payment inside the hold extends `payment_due_at` once, to the attempt's expiry (DV-11), so that a buyer who is part-way through the provider's page is not cancelled under them.
- **Mismatch** (FR-PAY-03). If the verified amount, currency or reference differs, the attempt becomes `succeeded_rejected`. A full refund is created and an `amount_mismatch` issue raised; the order stays unpaid.
- **Late payment.** If the order was already cancelled when a genuine success arrives, the payment is `succeeded_rejected`, refunded in full and raised as `late_payment`. The expiry job queries in-flight attempts before cancelling, which makes this rare.

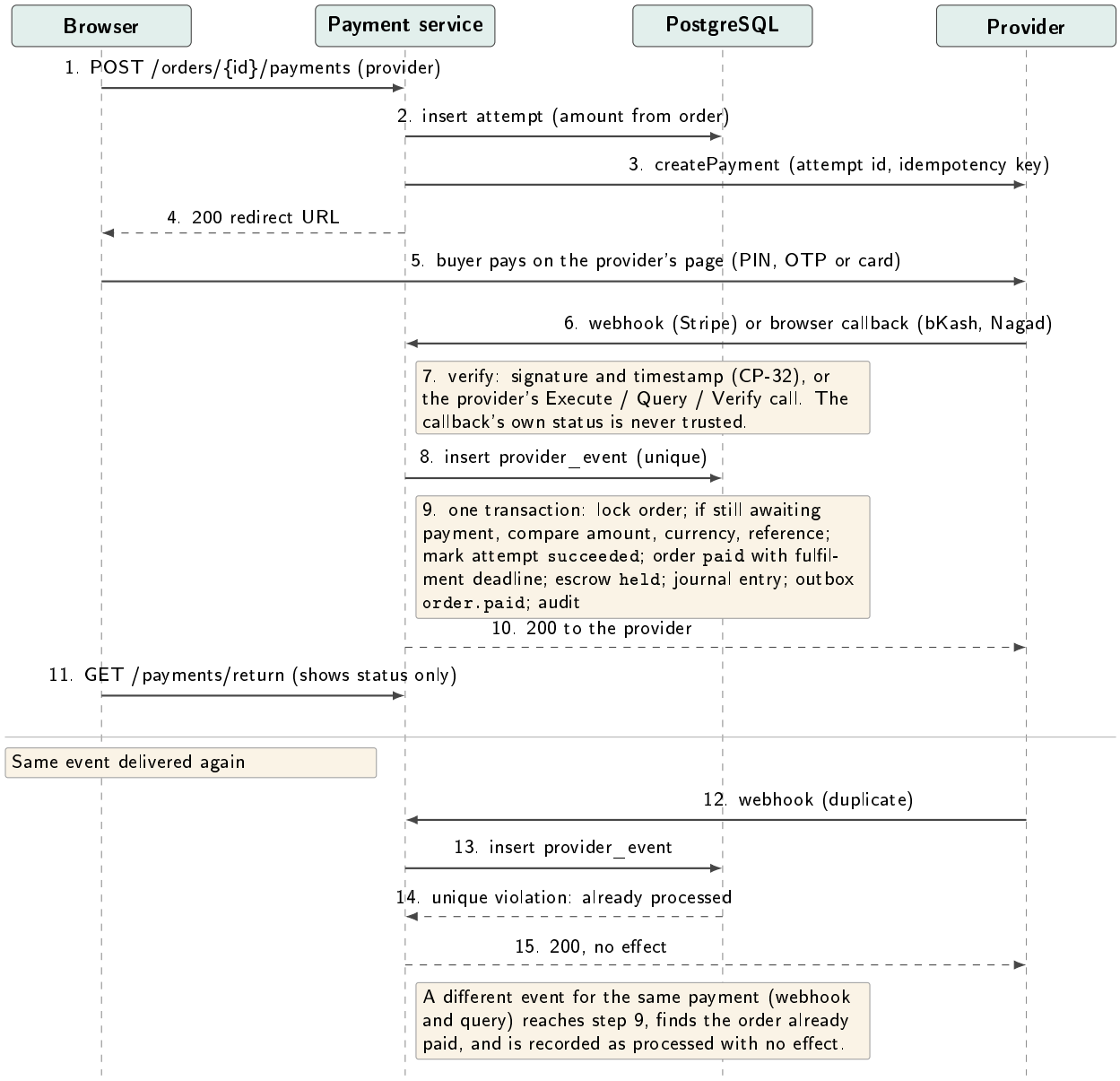


Figure 6: Payment confirmation and a duplicate delivery of the same event (FR-PAY-02 to FR-PAY-05).

- **Duplicate payment.** If the order is already paid by another attempt, the second one is `succeeded_duplicate` and refunded in full.
- **Failure.** A failed or abandoned attempt leaves the order awaiting payment, and the buyer may try again until `payment_due_at`. Provider calls are retried with exponential backoff and jitter using the same idempotency key (FR-PAY-16).

5.6 Fulfilment steps and the delivery event

Delivery and Game-Account report progress through `EscrowCommands`, inside their own transactions (DC-13):

- `recordFulfilmentStep` sets `fulfilment_done_at`. The steps are courier pickup and credential delivery. A meetup handover counts both as the step and as the delivery event.
- `recordDeliveryEvent` sets the event time and source (handover code, courier `delivered`, takeover complete) and the escrow's dispute-window and auto-release times. It writes `orders.order.delivered`.
- `refundCourierReturned` refunds the buyer in full, including the fee. Any courier charge is posted as `expense_courier_returns` (FR-DLV-12).

A step or event that arrives twice, or for an order in the wrong state, is ignored and logged. The first recorded time stands.

5.7 Release

Release happens by buyer confirmation after the delivery event (`confirm-receipt`); by the auto-release job at `auto_release_at` with no dispute open; or, for game accounts, by early release with step-up. Each runs one transaction:

1. lock the order and then the escrow (fixed lock order) and re-check the state;
2. compute the split with `item_refund` = 0;
3. set the escrow to `released` and write the history row;
4. insert the settlement record and post the release entry;
5. create the payout, choosing its status:
   - `held` if the seller is suspended for fraud;
   - `waiting_account` if there is no payout account;
   - `waiting_cooling_off` until the account is effective;
   - otherwise `pending`;
6. close the order, write `orders.escrow.released` to the outbox, and audit.

Because confirm-receipt, the release job, a dispute being opened and a cancellation all take the same escrow row lock, they are serialised. Whichever commits first decides, and the others find the new state and stop. Physical disputes close CP-19 − CP-18 (two days by default) before auto-release, so the two never compete. For game accounts both end at the same moment, and a dispute is accepted only strictly before `auto_release_at`.

5.8 Cancellation and deadline refunds

- **Before payment.** The expiry job cancels unpaid orders after `payment_due_at` (`checkout_expired` or `payment_window_expired`). Before cancelling, it queries any in-flight attempt, so a payment that succeeded without a callback is confirmed rather than lost. Listings returns the reserved stock on `order.cancelled` (FR-FIX-03).
- **After payment, before the delivery event** (FR-PAY-09). The buyer requests cancellation and the seller accepts, or the seller cancels unilaterally. Either way the

escrow becomes `refunded` for the full amount. A unilateral cancellation also emits `orders.order.cancelled_by_seller` for the strike.
- **Missed fulfilment deadline** (FR-PAY-08). The fulfilment job refunds every paid order whose required step is not recorded by `fulfilment_due_at` and that has no open dispute.
- **Stalled courier delivery.** If the item was picked up but no delivery event follows within CP-17 + CP-18, and no dispute is opened, the job raises a `stalled_delivery` issue for an operator rather than releasing or refunding automatically (DV-12).

5.9 Refunds

A refund row is created, with its own idempotency key, in the same transaction as the decision that causes it, together with its ledger entry. The refund worker then calls the provider's refund function on the original payment:

- success moves the refund to `succeeded` and posts the "refund paid out" entry;
- a failure is retried with backoff up to DV-13;
- after the retries are exhausted, or when the provider cannot refund (`unsupported`), the refund becomes `manual_required`, an operator issue is raised and the buyer is told when to expect the money (FR-PAY-12).

The database trigger guarantees that the refunds of one payment never add up to more than it.

5.10 Disputes

The Disputes unit calls `freezeForDispute`, accepted only if `EscrowQuery.disputeAllowed`: the escrow is `held` and the time is inside the window (FR-DSP-01, FR-DSP-11). This moves the escrow to `frozen_disputed`, which stops every automatic release (FR-DSP-02). `applyDisputeOutcome` executes the decision (FR-DSP-06):

- **release** works as Section 5.7;
- **full refund** refunds the whole item price, and the delivery fee too if the courier never picked up. If the courier did pick up, the fee stays with the courier and platform, so the escrow ends `partially_refunded` even though the buyer received everything refundable;
- **partial** applies the split with the decided item refund;
- **refund after return** leaves the escrow frozen until the Disputes unit reports the item back. A return fare paid by the buyer is deducted from the refund; one paid by the seller becomes a `seller_receivable`, recovered from later payouts.

`withdrawDispute` returns the escrow to `held` with `auto_release_at` = the later of the original time and now + 24 hours (FR-DSP-10). Decisions requiring four-eyes approval (CP-30) are approved in the Disputes unit before this call.

5.11 Payout accounts and payouts

- **Wallet.** An MFS wallet is accepted only if its number equals the seller's verified mobile number, from `IdentityQuery.getVerifiedPhone`. It enters `cooling_off` with `effective_at` = now + CP-29 (FR-PAY-17, FR-PAY-18).
- **Bank account.** A bank account enters `pending_review`. An operator compares the account name with the KYC name; reading that name through `getKycLegalName` is audited. Only after approval does the account go into cooling-off.
- **Change.** Adding or changing an account requires step-up and emits `payments.payout_account.changed`, so the seller's number and e-mail are alerted. The previous account becomes `replaced`. Payouts waiting for cooling-off are sent once the new account is effective.

- **Sending.** The payout worker sends `pending` payouts through the payout adapter with the payout's idempotency key, then posts the "payout sent" entry. Failures are retried and then raised as `payout_failed`.
- **Fraud hold.** When Unit 1 reports a suspension for fraud, the seller's pending payouts become `held` and new ones are created `held` (FR-TNS-05), until an operator lifts the hold.

5.12 Reconciliation and chargebacks

- **Every CP-46** (FR-PAY-15), the reconciliation job:
  - queries attempts that have been in flight longer than DV-15;
  - queries refunds sent but not confirmed;
  - checks paid orders whose attempt lacks a provider transaction id.

  Each query result is recorded as a provider event with a derived id, such as `query:{attempt}:{status}`, and processed through the normal idempotent path. A result that contradicts the ledger raises a `provider_disagrees` issue instead of being applied.
- **Daily**, where the provider offers a settlement report, the job compares the day's totals with `provider_clearing` postings.
- **Chargebacks** (FR-PAY-20, Could). A Stripe dispute event records a chargeback and posts it to `seller_receivable`. Later payouts to that seller are reduced until it is recovered, and an operator is notified.

5.13 Monetization

- **Commission and delivery margin** are taken in the split from the rates stored on the order (FR-MON-01 to FR-MON-03, FR-MON-10). There is no listing fee and no per-click charge (FR-MON-04).
- **Featured listing** (Could). The seller pays the CP-37 fee through a `featured_listing` attempt. On confirmation the purchase starts and `payments.featured.started` tells Listings to show the label; a job ends it on time.
- **Pro Seller** (Could). Card subscriptions use Stripe's recurring billing, confirmed by webhook. Because MFS wallets do not support recurring charges, MFS subscribers pay each month through a `subscription` attempt after a reminder. Two failed or missed renewals move the subscription to `lapsed` (FR-MON-08). `ProStatusQuery.isPro` is read when an order is created, so the rate applies only to orders created while the subscription is active.

5.14 Scheduled jobs

Each job selects due rows with `FOR UPDATE SKIP LOCKED` in batches, re-checks the row's state inside its transaction, and acts on the recorded deadline times. A job can therefore run on several instances at once, can be re-run safely, and catches up after downtime without acting twice (NFR-REL-03).

Job	Every	Action
Payment expiry	1 min	Query in-flight attempts; then cancel orders past `payment_due_at`.
Fulfilment deadline	1 min	Refund paid orders whose fulfilment step is overdue (FR-PAY-08).
Auto-release	1 min	Release held escrows past `auto_release_at` (FR-PAY-07b, FR-GAM-17).
Stalled delivery	1 hour	Raise `stalled_delivery` issues (DV-12).
Reconciliation	CP-46	As above (FR-PAY-15).

Job	Every	Action
Refund worker	30 s	Send pending refunds; retry; mark `manual_required` (FR-PAY-12).
Payout worker	1 min	Send payouts that are due; release cooled-off ones (FR-PAY-19).
Cancellation lapse	1 hour	Lapse pending requests at the delivery event or the fulfilment deadline.
Monetization	5 min	End featured periods; process renewals and lapses.
Outbox	5 s	Publish pending events.

6 Security Design

6.1 Verifying provider messages

- **Stripe webhooks.** The raw request body is verified against the `Stripe-Signature` header with the webhook secret of the current environment; the signed timestamp must be within CP-32. The event id is stored in `provider_events` before processing, so a replay is a unique violation and has no effect (SEC-PAY-03, SEC-PAY-04).
- **bKash and Nagad callbacks.** These arrive through the buyer's browser and are untrusted. The callback URL contains the attempt id and an HMAC of it ("signed state"), which stops a forged callback from naming another attempt. In any case the server never acts on the callback's content: it calls the provider's execute, query or verify API with its own credentials and acts only on that answer.
- **Every confirmation**, whatever its route, must match the attempt: amount, currency BDT and merchant reference (FR-PAY-03). The amount is computed on the server; any amount from the client is ignored (SEC-PAY-06).

6.2 Card and wallet data

Card numbers are entered only on Stripe's hosted Checkout page, reached by redirect, so TrustBazaar's pages contain no card form and no payment script. This keeps the platform within SAQ A (SEC-PAY-01, NFR-CMP-03). MFS PINs and OTPs are entered only on bKash's or Nagad's pages (SEC-PAY-02). The database stores only provider ids and references, and the last four digits of a bank account; the full bank account number is encrypted with Unit 1's field key.

6.3 Credentials, environments and idempotency

- Provider credentials are read from environment variables. At start-up the backend checks that every credential matches the configured mode, and refuses to start on a mismatch (SEC-PAY-05). For example, Stripe keys must begin `sk_test_` in sandbox mode and `sk_live_` only in production.
- Every attempt, refund and payout has its own idempotency key, stored before the provider is called, and sent as Stripe's `Idempotency-Key` header or as the merchant invoice or order number for bKash and Nagad (SEC-PAY-07). A retry after a timeout therefore cannot create a second charge, refund or payout. Before retrying a bKash or Nagad refund whose outcome is unknown, the worker queries the provider first.

6.4 Abuse and fraud controls

- Order limits CP-27 for newly verified buyers (FR-TNS-07); checkout and payment start are rate-limited (SEC-ABU-01).
- Payouts go only to a wallet on the seller's own verified number, or to a bank account in the seller's KYC name. A changed account waits CP-29 before receiving money, and both contact channels are alerted. These defeat the common pattern of taking over an account and redirecting its payouts.
- Fraud holds stop payouts while Trust & Safety investigates (FR-TNS-05).
- Release before the delivery event is impossible. Scammers often press buyers to "confirm now", and the system cannot do so.

6.5 Operators

Operators can resolve reconciliation issues, approve bank accounts and lift payout holds, each with a recorded reason and an audit entry. They cannot move money in any other way: there is no endpoint to edit an order, an escrow or a ledger entry, and the database refuses such edits from the application role (SRS C-5, FR-OPS-02).

7 Parameters

SRS parameters used: CP-10, CP-11 (payment windows); CP-13 (checkout hold); CP-15, CP-16, CP-17 (fulfilment deadlines); CP-18, CP-19 (dispute window, auto-release); CP-22, CP-24 (game account); CP-25, CP-26 (fees); CP-27 (order limits); CP-29 (payout cooling-off); CP-30 (four-eyes, applied in Disputes); CP-32 (webhook tolerance and event retention); CP-37 (featured fees); CP-46 (reconciliation interval).

Design values introduced (proposed for the SRS parameter table, continuing Unit 1's DV-1 to DV-10):

ID	Value	Default	Reason
DV-11	Payment attempt lifetime (and hold extension)	15 minutes	Time on the provider's page; extends the hold once.
DV-12	Stalled courier delivery review	CP-17 + CP-18 after pickup	Hands the case to a person rather than guessing.
DV-13	Refund retries before manual handling	8 attempts over 24 hours	Rides out provider outages.
DV-14	Payout retries before raising an issue	6 attempts over 12 hours	As above.
DV-15	In-flight attempt age before reconciliation queries it	2 minutes	Most callbacks arrive within seconds.
DV-16	Cancellation request lapse	At the delivery event or fulfilment deadline	No request outlives the order's next step.

8 Testing

Level	Cases
Database (done)	45 checks in `02_orders_payments_tests.sql`. Each rejection is required to name the intended constraint, so no check can pass because a different rule fired. They include the SRS worked example, its partial-refund variant and 20,000 random splits. A concurrency test sends 15 simultaneous confirmations of one payment and produces exactly one escrow (Appendix B.1).
Unit	`SettlementService` against `payments.split()` on random inputs (identical results); deadline arithmetic from stored parameter versions; each adapter's `verify` rejects a bad signature, a stale timestamp and a mismatched amount; the payout-status choice.
Integration	50 simultaneous checkouts for the last unit, with a Listings stub, give exactly one reservation (AC-2). Confirm-receipt racing auto-release and dispute opening gives exactly one outcome. Expiry with an in-flight attempt confirms rather than cancels. A late payment is refunded; a duplicate payment is refunded. A refund retried after a timeout does not refund twice. The outbox survives a crash between commit and publication.
Sandbox contract	For each provider in its sandbox: pay, cancel on the provider page, refund, partial refund, replayed webhook, forged callback.
Acceptance	AC-2, AC-3, AC-4 and AC-6 (SRS Appendix A), each automated under the reference load (NFR-PERF-08).

9 Notes for Integration

I-8 Unit 1 port addition. `IdentityQuery.getPrincipal` should also return `verifiedAt` for the CP-27 limits, and Unit 1 should emit a fraud flag with `identity.account.suspended` so that payouts can be held.

I-9 Listings (Unit 3) must provide `reserve` and `release` as single conditional updates (DC-13), mark units sold on `order.paid` and release them on `order.cancelled`.

I-10 Delivery (Unit 4) issues quotes bound to buyer, listing and expiry, and reports pickup, handover code, delivered and returned through `EscrowCommands`.

I-11 Game-Account (Unit 5) reports credential delivery and takeover completion, and exposes `assertCheckoutReady`.

I-12 Disputes (Unit 7) owns dispute windows by type, four-eyes approval of large outcomes, and the return shipment for "refund after return".

I-13 SRS clarifications proposed:

- a "full refund" outcome after courier pickup ends `partially_refunded`, because the delivery fee is not refundable;
- add DV-11 to DV-16 to the SRS parameter table;
- MFS subscriptions renew by monthly payment, not automatic charge.

A Traceability

Requirement	Design element	Section
`FR-PAY-01`	Three adapters behind one interface; sandbox and test modes	3.2
`FR-PAY-02`	Confirmation only from webhook signature or provider query; return page shows status only	5.5, 6.1
`FR-PAY-03`	Amount, currency and reference compared; mismatch gives `succeeded_rejected` and a refund	5.5
`FR-PAY-04`	Unique provider events; one successful attempt per order; concurrency test	5.5, 4
`FR-PAY-05`	Escrow row created `held` with the full amount	5.5

Requirement	Design element	Section
FR-PAY-06	`escrow_transition` trigger; Figure 4	4
FR-PAY-07	Release paths and shared escrow lock; no confirm before the delivery event	5.7
FR-PAY-08	Fulfilment step and deadline columns; fulfilment job	5.2, 5.8
FR-PAY-09	Cancellation requests; seller cancellation with strike event	5.8
FR-PAY-10	`escrow_history`, append-only	4
FR-PAY-11	`payments.split()`; item refund first; commission on released part	4.4
FR-PAY-12	Refund rows, worker, retries, `manual_required`	5.9
FR-PAY-13	`payments.settlements` written with the terminal transition	4.3
FR-PAY-14	bigint poisha; half-up rounding; sums checked by constraint; 20,000-case property test	4.4
FR-PAY-15	Reconciliation job and issues queue	5.12
FR-PAY-16	Backoff with jitter reusing idempotency keys	5.5
FR-PAY-17	Wallet equals verified number; bank name check by operator	5.11
FR-PAY-18	Step-up; alert event; cooling-off `effective_at`	5.11
FR-PAY-19	Payouts table and worker; simulated adapter	5.11
FR-PAY-20	Chargebacks posted to `seller_receivable`	5.12
FR-PAY-21	Receipt and statement endpoints from settlements	3.1
FR-FIX-01	Verified(buy); buyer $\neq$ seller constraint	5.3
FR-FIX-02	`ListingsPort.reserve` in the order transaction (DC-13)	5.3
FR-FIX-03	Payment expiry job; `order.cancelled` returns stock	5.8
FR-FIX-04	`order.paid` lets Listings mark units sold	5.3
FR-FIX-05	Single conditional update gives a definite answer per request	5.3
FR-AUC-14	`OrdersPort.createAuctionOrder`; window CP-10; one live order per auction	5.4
FR-AUC-15	Expiry emits `payment_expired`; second-chance orders with CP-11	5.4
FR-DLV-11	Pickup deadline CP-16; stalled-delivery review	5.8
FR-DLV-12	`refundCourierReturned`; courier return expense	5.6
FR-DLV-13	Margin and courier payout in the split	4.4
FR-GAM-10	Credential delivery deadline CP-22 refunds	5.2, 5.8
FR-GAM-17	`game_protection` policy; early release with step-up	5.2, 5.7
FR-DSP-02	`freezeForDispute`; frozen escrow names its dispute	5.10
FR-DSP-06	`applyDisputeOutcome`: release, full, partial, after return	5.10
FR-DSP-10	`withdrawDispute` with the later of the two release times	5.10
FR-DSP-11	`disputeAllowed` requires `held` and the window	5.10
FR-MON-01	Commission in the split from the rate stored on the order	4.4, 5.13
FR-MON-02	`ProStatusQuery` read at order creation	5.13
FR-MON-03	Delivery margin in the split	4.4
FR-MON-04	No listing or click fees exist in the design	5.13
FR-MON-05	Featured purchases through attempts; started and ended events	5.13
FR-MON-06	Subscriptions table; Stripe recurring, MFS monthly	5.13
FR-MON-07	Subscription payments confirmed like purchases	5.13
FR-MON-08	Lapse after failed renewals; event to Listings	5.13
FR-MON-09	`payments.revenue_log` view	4.3
FR-MON-10	Rates and parameter version stored on the order	5.3
FR-TNS-05	Payout holds on fraud suspension	5.11
FR-TNS-07	CP-27 checked at checkout	5.3
FR-ACC-11	`AccountClosureCheck` implemented by this unit	3.1
SEC-PAY-01	Stripe Checkout by redirect: SAQ A	6.1
SEC-PAY-02	PINs and OTPs only on provider pages; references only	6.1
SEC-PAY-03	Signature check, or the provider's own query	6.1
SEC-PAY-04	Timestamp tolerance; unique provider events	6.1
SEC-PAY-05	Start-up check of credential mode	6.1
SEC-PAY-06	Server-computed amounts; client amounts ignored	6.1
SEC-PAY-07	Idempotency keys on attempts, refunds, payouts	6.1
IF-SW-01	bKash adapter	3.2
IF-SW-02	Nagad adapter	3.2
IF-SW-03	Stripe adapter	3.2

Requirement	Design element	Section
IF-UI-04	Checkout endpoint returns full price breakdown	3.1
IF-UI-05	Order view with escrow state and deadlines	3.1, 5.2
IF-UI-08	Payout account, payouts, statements endpoints	3.1
NFR-PERF-06	Confirmation in one transaction after verification	5.5
NFR-PERF-08	Database barriers hold under concurrency (tests)	B.1
NFR-REL-02	Acknowledge only after commit	5.5
NFR-REL-03	Jobs with SKIP LOCKED acting on stored times	5.14
NFR-REL-04	Hold extension; outage handling through expiry queries	5.5
NFR-CMP-02	Ledger maps to a licensed provider in production	4.3
NFR-CMP-03	SAQ A scope	6.1
NFR-CMP-05	Payouts marked simulated; sandbox modes	3.2
DR-03	`listing_snapshot` on every order	4
DR-05	Orders, escrow history, settlements, refunds, payouts, provider references, events	4
DR-10	Append-only histories and balanced ledger	4
SEC-LOG-01	Every money action audited in its transaction (DC-8); escrow history	5.7, 4

B Database Schema and Tests

B.1 Test results

Loaded after the Unit 1 schema in PostgreSQL 16, using PostgreSQL 15 features only. Every check passed. The concurrency test is listed after the script output.

```
PASS  FR-FIX-01 buyer cannot buy own listing [buyer_not_seller]
PASS  total must equal item price plus delivery fee [total_matches]
PASS  FR-LST-05 auction order has quantity 1 [auction_single_unit]
PASS  FR-DLV-02 self-pickup adds no delivery fee [fee_only_for_courier]
PASS  FR-GAM-06 a game account cannot go by courier [game_fulfilment_matches]
PASS  an order cannot be paid before a fulfilment method is chosen [paid_needs_fulfilment]
PASS  a paid order must carry its fulfilment deadline (FR-PAY-08) [paid_needs_deadline]
PASS  FR-AUC-14 one live order per auction [orders_one_live_auction]
PASS  FR-AUC-15 second-chance order allowed after the winner's order is cancelled
PASS  AC-4 an order can be paid only once [attempts_one_success_per_order]
PASS  a second successful payment is recorded as a duplicate (to be refunded)
PASS  only BDT is accepted [attempts_currency_check]
PASS  SEC-PAY-04 a replayed provider event is rejected [provider_events_provider_event_id_key]
PASS  FR-PAY-05 escrow must start in held [must start in held]
PASS  FR-PAY-06 held -> partially_refunded is not a permitted transition [is not permitted]
PASS  FR-DSP-02 a frozen escrow names its dispute [escrow_frozen_has_dispute]
PASS  amount collected cannot change [amount_collected is fixed]
PASS  a delivery event needs the fulfilment step first [delivery_after_fulfilment]
PASS  dispute window must end before auto-release [escrow_window_before_release]
PASS  a partial refund must leave something released [escrow_partial_both]
PASS  a terminal escrow cannot change again [is not permitted]
PASS  FR-PAY-10 escrow history is append-only [append-only]
PASS  SRS worked example, release: 500 + 15 + 135 + 9,500 = 10,150 BDT
PASS  SRS worked example, partial refund of BDT 2,000: commission 400, seller 7,600
PASS  courier never picked up: the whole delivery fee is refunded
PASS  half-up rounding to the poisha: 5% of 33 poisha is 2 (1.65 rounds up)
PASS  FR-PAY-14 property: 20,000 random cases always add up exactly and never go negative
PASS  item refund larger than the item price is refused [out of range]
PASS  FR-PAY-13 one settlement record per order, never rewritten [append-only]
PASS  FR-PAY-14 a settlement whose parts do not add up is refused [settlement_parts_sum]
PASS  refunds can never exceed the payment [exceed payment]
PASS  an unbalanced journal entry is refused at commit [does not balance]
PASS  balanced entries are accepted
PASS  escrow liability of a settled order nets to zero
PASS  FR-MON-09 revenue log shows commission 400 and margin 15 BDT
PASS  ledger balance equals settlement record
PASS  journal lines are append-only [append-only]
PASS  FR-PAY-17 one current payout account per seller [payout_account_one_current]
PASS  FR-PAY-17 a bank account becomes usable only after an operator check [bank_reviewed]
PASS  one payout per order [payouts_order_id_key]
```

```
PASS  FR-TNS-05 a held payout records why [payout_hold_reason]
PASS  outbox events belong to this unit [outbox_event_type_check]
PASS  application role cannot rewrite escrow history [permission denied]
PASS  application role cannot change the ledger chart of accounts [permission denied]
PASS  application role cannot delete orders [permission denied]

Concurrency test: 15 simultaneous confirmations of one payment
(10 deliveries of the same webhook event + 5 different confirmation events):
  escrows created = 1, successful attempts = 1, provider events recorded = 1, order status = paid
```

B.2 Schema

```
-- ================================================================
-- TrustBazaar - Unit 2: Order, Payment & Escrow - PostgreSQL 15 schema
-- Design document: docs/design/02_orders_payments.pdf
-- Requirements: SRS v2.0 FR-PAY-01..21, FR-FIX-01..05 (order side),
--   FR-AUC-14/15 (order side), FR-DLV-11..13 and FR-GAM-10/17 (escrow
--   side), FR-DSP-02/06/10/11 (escrow side), FR-MON-01..10, FR-TNS-05/07,
--   SEC-PAY-01..07.
-- Depends on: 01_identity_access.sql (identity.users, identity.staff_accounts,
--   identity.forbid_change, audit.actor_type).
-- Conventions as Unit 1 (DC-4): random UUID keys, timestamptz, money as
-- bigint poisha (1 BDT = 100 poisha), invariants enforced in the database.
-- References to later units (listings, delivery, disputes) are plain UUIDs
-- without foreign keys (DC-5).
-- ================================================================

CREATE SCHEMA IF NOT EXISTS orders;
CREATE SCHEMA IF NOT EXISTS payments;

-- ---------------------------------------------------------------- types
CREATE TYPE orders.order_kind     AS ENUM ('fixed_price', 'auction', 'second_chance');
CREATE TYPE orders.item_kind      AS ENUM ('physical', 'game_account');
CREATE TYPE orders.fulfilment     AS ENUM ('self_pickup', 'courier', 'game_account');
CREATE TYPE orders.order_status   AS ENUM ('awaiting_payment', 'cancelled', 'paid', 'closed');
CREATE TYPE orders.cancel_reason  AS ENUM
  ('checkout_expired', 'payment_window_expired', 'buyer_abandoned', 'listing_withdrawn');
CREATE TYPE orders.escrow_state   AS ENUM
  ('held', 'frozen_disputed', 'released', 'refunded', 'partially_refunded');      -- FR-PAY-06
CREATE TYPE orders.release_policy AS ENUM ('standard', 'game_protection');      -- FR-GAM-17
CREATE TYPE orders.fulfilment_step AS ENUM
  ('meetup_handover', 'courier_pickup', 'credentials_delivered');               -- FR-PAY-08
CREATE TYPE orders.delivery_source AS ENUM
  ('handover_code', 'courier_delivered', 'game_takeover');                     -- delivery event
CREATE TYPE orders.cancel_request_status AS ENUM ('pending', 'accepted', 'declined', 'lapsed');

CREATE TYPE payments.provider       AS ENUM ('bkash', 'nagad', 'stripe');
CREATE TYPE payments.purpose        AS ENUM ('order', 'featured_listing', 'subscription');
CREATE TYPE payments.attempt_status AS ENUM
  ('created', 'redirected', 'succeeded', 'failed', 'cancelled', 'expired',
   'succeeded_duplicate',    -- a second successful payment for an already-paid order: refunded
   'succeeded_rejected');    -- succeeded at the provider but not accepted (mismatch or late): refunded
CREATE TYPE payments.refund_reason AS ENUM
  ('fulfilment_deadline', 'cancellation', 'dispute_full', 'dispute_partial', 'courier_returned',
   'duplicate_payment', 'amount_mismatch', 'late_payment', 'fee_refund');
CREATE TYPE payments.refund_status AS ENUM ('pending', 'sent', 'succeeded', 'failed', 'manual_required');
CREATE TYPE payments.payout_account_kind   AS ENUM ('bkash_wallet', 'nagad_wallet', 'bank');
CREATE TYPE payments.payout_account_status AS ENUM
  ('pending_review', 'cooling_off', 'active', 'rejected', 'replaced');
CREATE TYPE payments.payout_status AS ENUM
  ('waiting_account', 'waiting_cooling_off', 'held', 'pending', 'sent', 'failed');
CREATE TYPE payments.entry_kind AS ENUM
  ('payment_received', 'escrow_release', 'escrow_refund', 'escrow_partial', 'refund_paid',
   'payout_sent', 'courier_settled', 'fee_received', 'courier_return_charge',
   'chargeback', 'chargeback_recovery');

-- ================================================================
-- Orders (FR-FIX, FR-AUC-14/15, FR-MON-10, DR-03)
-- ================================================================
CREATE TABLE orders.orders (
  id                    uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  kind                  orders.order_kind NOT NULL,
  listing_id            uuid NOT NULL,                        -- Listings unit (no FK, DC-5)
```

```
  item_kind              orders.item_kind NOT NULL,
  buyer_id               uuid NOT NULL REFERENCES identity.users(id),
  seller_id              uuid NOT NULL REFERENCES identity.users(id),
  quantity               integer NOT NULL CHECK (quantity >= 1),
  unit_price             bigint  NOT NULL CHECK (unit_price > 0),
  item_price             bigint  NOT NULL,
  fulfilment             orders.fulfilment,                    -- chosen before payment
  delivery_quote_id      uuid,                                 -- Delivery unit quote, fixed at checkout
  delivery_fee           bigint  NOT NULL DEFAULT 0 CHECK (delivery_fee >= 0),
  total                  bigint  NOT NULL,
  -- snapshot of the listing and of the fee rules when the order was created (DR-03, FR-MON-10, FR-OPS-08)
  listing_snapshot       jsonb   NOT NULL,
  commission_bps         integer NOT NULL CHECK (commission_bps BETWEEN 0 AND 5000),     -- 500 = 5 %
  delivery_margin_bps    integer NOT NULL CHECK (delivery_margin_bps BETWEEN 0 AND 5000),
  params_version         integer NOT NULL,
  status                 orders.order_status NOT NULL DEFAULT 'awaiting_payment',
  payment_due_at         timestamptz NOT NULL,                 -- CP-13 checkout hold or CP-10/CP-11 window
  paid_at                timestamptz,
  fulfilment_step        orders.fulfilment_step,               -- which step must happen by fulfilment_due_at
  fulfilment_due_at      timestamptz,
  fulfilment_done_at     timestamptz,
  delivery_event_at      timestamptz,
  delivery_event_source  orders.delivery_source,
  cancelled_at           timestamptz,
  cancel_reason          orders.cancel_reason,
  closed_at              timestamptz,
  created_at             timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT buyer_not_seller CHECK (buyer_id <> seller_id),                                    --
    FR-FIX-01, FR-AUC-02
  CONSTRAINT item_price_matches CHECK (item_price = unit_price * quantity),
  CONSTRAINT total_matches CHECK (total = item_price + delivery_fee),
  CONSTRAINT auction_single_unit CHECK (kind = 'fixed_price' OR quantity = 1),
    -- FR-LST-05
  -- fee only for courier (FR-DLV-02, FR-GAM-06)
  CONSTRAINT fee_only_for_courier CHECK (fulfilment IS NOT DISTINCT FROM 'courier' OR delivery_fee = 0),
  CONSTRAINT courier_needs_quote CHECK (fulfilment IS DISTINCT FROM 'courier' OR delivery_quote_id IS NOT
    NULL),
  -- game accounts are fulfilled only by the game-account handover, and only they are
  CONSTRAINT game_fulfilment_matches CHECK (fulfilment IS NULL OR ((item_kind = 'game_account') = (fulfilment
     = 'game_account'))),
  -- payment requires a chosen fulfilment method
  CONSTRAINT paid_needs_fulfilment CHECK (status IN ('awaiting_payment', 'cancelled') OR fulfilment IS NOT
    NULL),
  CONSTRAINT paid_at_matches_status CHECK ((status IN ('paid', 'closed')) = (paid_at IS NOT NULL)),
  CONSTRAINT cancel_fields CHECK ((status = 'cancelled') = (cancelled_at IS NOT NULL AND cancel_reason IS NOT
     NULL)),
  CONSTRAINT closed_fields CHECK ((status = 'closed') = (closed_at IS NOT NULL)),
  CONSTRAINT paid_needs_deadline CHECK (paid_at IS NULL OR (fulfilment_step IS NOT NULL AND fulfilment_due_at
     IS NOT NULL)),
  CONSTRAINT done_after_paid CHECK (fulfilment_done_at IS NULL OR paid_at IS NOT NULL),
  CONSTRAINT delivery_event_fields CHECK ((delivery_event_at IS NULL) = (delivery_event_source IS NULL)),
  CONSTRAINT delivery_after_fulfilment CHECK (delivery_event_at IS NULL OR fulfilment_done_at IS NOT NULL)
);
CREATE INDEX orders_buyer   ON orders.orders (buyer_id, created_at DESC);
CREATE INDEX orders_seller  ON orders.orders (seller_id, created_at DESC);
CREATE INDEX orders_listing ON orders.orders (listing_id);
-- scheduled jobs (NFR-REL-03): each scans only rows that are due
CREATE INDEX orders_due_payment     ON orders.orders (payment_due_at)    WHERE status = 'awaiting_payment';
CREATE INDEX orders_due_fulfilment  ON orders.orders (fulfilment_due_at) WHERE status = 'paid' AND
    fulfilment_done_at IS NULL;
-- an auction listing has at most one live (unpaid or paid) order (FR-AUC-14/15)
CREATE UNIQUE INDEX orders_one_live_auction ON orders.orders (listing_id)
  WHERE kind IN ('auction', 'second_chance') AND status IN ('awaiting_payment', 'paid', 'closed');

-- ====================================================================
-- Escrow (FR-PAY-05..11, Table 4.8 of the SRS)
-- ====================================================================
CREATE TABLE orders.escrows (
  order_id          uuid PRIMARY KEY REFERENCES orders.orders(id),
  state             orders.escrow_state NOT NULL DEFAULT 'held',
  release_policy    orders.release_policy NOT NULL,
  amount_collected  bigint NOT NULL CHECK (amount_collected > 0),
  amount_released   bigint NOT NULL DEFAULT 0 CHECK (amount_released >= 0),   -- to seller, courier and
    platform
  amount_refunded   bigint NOT NULL DEFAULT 0 CHECK (amount_refunded >= 0),   -- to buyer
```

```
  dispute_window_ends_at timestamptz,          -- delivery event + CP-18 (physical) / + CP-24 (game)
  auto_release_at   timestamptz,               -- delivery event + CP-19 (physical) / + CP-24 (game)
  open_dispute_id   uuid,                      -- Disputes unit (no FK)
  created_at        timestamptz NOT NULL DEFAULT now(),
  terminal_at       timestamptz,
  CONSTRAINT escrow_amount_cap CHECK (amount_released + amount_refunded <= amount_collected),
  -- amounts match the state
  CONSTRAINT escrow_open_no_amounts CHECK (state NOT IN ('held', 'frozen_disputed') OR (amount_released = 0
    AND amount_refunded = 0)),
  CONSTRAINT escrow_terminal_sums CHECK (state NOT IN ('released', 'refunded', 'partially_refunded')
        OR amount_released + amount_refunded = amount_collected),
  CONSTRAINT escrow_released_no_refund CHECK (state <> 'released' OR amount_refunded = 0),
  CONSTRAINT escrow_refunded_no_release CHECK (state <> 'refunded' OR amount_released = 0),
  CONSTRAINT escrow_partial_both CHECK (state <> 'partially_refunded' OR (amount_released > 0 AND
    amount_refunded > 0)),
  CONSTRAINT escrow_terminal_time CHECK ((state IN ('released', 'refunded', 'partially_refunded')) = (
    terminal_at IS NOT NULL)),
  CONSTRAINT escrow_frozen_has_dispute CHECK ((state = 'frozen_disputed') = (open_dispute_id IS NOT NULL)),
  CONSTRAINT escrow_window_before_release CHECK (dispute_window_ends_at IS NULL OR auto_release_at IS NULL OR
     dispute_window_ends_at <= auto_release_at)
);
CREATE INDEX escrow_auto_release ON orders.escrows (auto_release_at) WHERE state = 'held' AND auto_release_at
     IS NOT NULL;

-- permitted transitions only (FR-PAY-06); any other is refused
CREATE FUNCTION orders.check_escrow_transition() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.state <> 'held' THEN
      RAISE EXCEPTION 'escrow must start in held' USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
  END IF;
  IF NEW.state IS DISTINCT FROM OLD.state THEN
    IF NOT ((OLD.state, NEW.state) IN (
        ('held'::orders.escrow_state, 'released'::orders.escrow_state),
        ('held',            'refunded'),
        ('held',            'frozen_disputed'),
        ('frozen_disputed', 'released'),
        ('frozen_disputed', 'refunded'),
        ('frozen_disputed', 'partially_refunded'),
        ('frozen_disputed', 'held'))) THEN
      RAISE EXCEPTION 'escrow % -> % is not permitted', OLD.state, NEW.state
        USING ERRCODE = 'check_violation';
    END IF;
  END IF;
  IF OLD.state IN ('released', 'refunded', 'partially_refunded') THEN
    RAISE EXCEPTION 'escrow is terminal' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.amount_collected <> OLD.amount_collected THEN
    RAISE EXCEPTION 'amount_collected is fixed' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER escrow_transition BEFORE INSERT OR UPDATE ON orders.escrows
  FOR EACH ROW EXECUTE FUNCTION orders.check_escrow_transition();

CREATE TABLE orders.escrow_history (                                    -- FR-PAY-10, append-only
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id      uuid NOT NULL REFERENCES orders.orders(id),
  from_state    orders.escrow_state,                    -- NULL for creation
  to_state      orders.escrow_state NOT NULL,
  actor_type    audit.actor_type NOT NULL,
  actor_id      uuid,
  reason        text NOT NULL,
  amount_released bigint NOT NULL DEFAULT 0,
  amount_refunded bigint NOT NULL DEFAULT 0,
  related_id    uuid,                                   -- dispute, approval, refund ...
  at            timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX escrow_hist_order ON orders.escrow_history (order_id, at);
CREATE TRIGGER escrow_history_append_only BEFORE UPDATE OR DELETE ON orders.escrow_history
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

CREATE TABLE orders.cancellation_requests (                             -- FR-PAY-09
```

```
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id      uuid NOT NULL REFERENCES orders.orders(id),
  requested_by  uuid NOT NULL REFERENCES identity.users(id),
  reason        varchar(500) NOT NULL,
  status        orders.cancel_request_status NOT NULL DEFAULT 'pending',
  created_at    timestamptz NOT NULL DEFAULT now(),
  decided_at    timestamptz,
  CHECK ((status = 'pending') = (decided_at IS NULL))
);
CREATE UNIQUE INDEX cancel_one_pending ON orders.cancellation_requests (order_id) WHERE status = 'pending';

CREATE TABLE orders.outbox (                                              -- DC-6
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type     varchar(80) NOT NULL CHECK (event_type ~ '^(orders|payments)\.[a-z_]+\.[a-z_]+$'),
  aggregate_id   uuid NOT NULL,
  payload        jsonb NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now(),
  attempts       smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at  timestamptz
);
CREATE INDEX orders_outbox_pending ON orders.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ================================================================
-- Payment attempts and provider events (FR-PAY-01..04, SEC-PAY-03/04/07)
-- ================================================================
CREATE TABLE payments.attempts (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),   -- also the merchant reference sent to
    the provider
  purpose           payments.purpose NOT NULL,
  order_id          uuid REFERENCES orders.orders(id),
  subject_id        uuid,                                         -- featured listing or subscription
  payer_id          uuid NOT NULL REFERENCES identity.users(id),
  provider          payments.provider NOT NULL,
  amount            bigint NOT NULL CHECK (amount > 0),           -- computed on the server (SEC-PAY-06)
  currency          char(3) NOT NULL DEFAULT 'BDT' CHECK (currency = 'BDT'),
  idempotency_key   uuid NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  provider_payment_id  varchar(120),
  provider_txn_id   varchar(120),                                 -- needed for refunds
  status            payments.attempt_status NOT NULL DEFAULT 'created',
  created_at        timestamptz NOT NULL DEFAULT now(),
  expires_at        timestamptz NOT NULL,
  confirmed_at      timestamptz,
  failure_code      varchar(60),
  CHECK ((purpose = 'order') = (order_id IS NOT NULL)),
  CHECK (purpose = 'order' OR subject_id IS NOT NULL),
  CHECK ((status IN ('succeeded', 'succeeded_duplicate', 'succeeded_rejected')) = (confirmed_at IS NOT NULL))
);
CREATE UNIQUE INDEX attempts_provider_ref ON payments.attempts (provider, provider_payment_id) WHERE
    provider_payment_id IS NOT NULL;
-- acceptance criterion 4: an order can be paid only once
CREATE UNIQUE INDEX attempts_one_success_per_order ON payments.attempts (order_id) WHERE status = 'succeeded'
    ;
CREATE INDEX attempts_inflight ON payments.attempts (created_at) WHERE status IN ('created', 'redirected');

CREATE TABLE payments.provider_events (                                   -- SEC-PAY-04
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  provider       payments.provider NOT NULL,
  event_id       varchar(160) NOT NULL,          -- provider's event id, or a derived id for query results
  event_type     varchar(80)  NOT NULL,
  verified_by    varchar(20)  NOT NULL CHECK (verified_by IN ('signature', 'query')),
  attempt_id     uuid REFERENCES payments.attempts(id),
  payload_sha256 bytea NOT NULL CHECK (octet_length(payload_sha256) = 32),
  received_at    timestamptz NOT NULL DEFAULT now(),
  processed_at   timestamptz,
  UNIQUE (provider, event_id)                    -- a replayed event cannot be processed twice
);

-- ================================================================
-- Refunds (FR-PAY-11/12)
-- ================================================================
CREATE TABLE payments.refunds (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id        uuid NOT NULL REFERENCES payments.attempts(id),   -- refunded to the original payment
  order_id          uuid REFERENCES orders.orders(id),
```

```
  amount             bigint NOT NULL CHECK (amount > 0),
  reason             payments.refund_reason NOT NULL,
  idempotency_key    uuid NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  status             payments.refund_status NOT NULL DEFAULT 'pending',
  provider_refund_id varchar(120),
  attempts_made      smallint NOT NULL DEFAULT 0,
  next_attempt_at    timestamptz NOT NULL DEFAULT now(),
  created_at         timestamptz NOT NULL DEFAULT now(),
  completed_at       timestamptz,
  CHECK ((status = 'succeeded') = (completed_at IS NOT NULL))
);
CREATE INDEX refunds_due ON payments.refunds (next_attempt_at) WHERE status IN ('pending', 'failed');

-- the total refunded from one payment can never exceed that payment
CREATE FUNCTION payments.check_refund_cap() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE paid bigint; refunded bigint;
BEGIN
  SELECT amount INTO paid FROM payments.attempts WHERE id = NEW.attempt_id FOR UPDATE;
  SELECT coalesce(sum(amount), 0) INTO refunded FROM payments.refunds
   WHERE attempt_id = NEW.attempt_id AND id <> NEW.id AND status <> 'manual_required';
  IF refunded + NEW.amount > paid THEN
    RAISE EXCEPTION 'refunds␣(%␣+␣%)␣exceed␣payment␣%', refunded, NEW.amount, paid
      USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER refund_cap BEFORE INSERT OR UPDATE OF amount ON payments.refunds
  FOR EACH ROW EXECUTE FUNCTION payments.check_refund_cap();

-- ================================================================
-- Payout accounts and payouts (FR-PAY-17..19, FR-TNS-05)
-- ================================================================
CREATE TABLE payments.payout_accounts (
  id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id            uuid NOT NULL REFERENCES identity.users(id),
  kind               payments.payout_account_kind NOT NULL,
  wallet_e164        varchar(16) CHECK (wallet_e164 ~ '^\+8801[3-9][0-9]{8}$'),
  bank_name          varchar(80),
  branch_name        varchar(80),
  routing_number     varchar(9)  CHECK (routing_number ~ '^[0-9]{9}$'),
  account_name       varchar(120),
  account_number_enc bytea,                                  -- AES-256-GCM (Unit 1 field key)
  account_last4      char(4),
  status             payments.payout_account_status NOT NULL,
  effective_at       timestamptz,                            -- end of cooling-off CP-29
  reviewed_by        uuid REFERENCES identity.staff_accounts(id),
  reviewed_at        timestamptz,
  created_at         timestamptz NOT NULL DEFAULT now(),
  replaced_at        timestamptz,
  CONSTRAINT bank_fields CHECK ((kind = 'bank') = (bank_name IS NOT NULL AND routing_number IS NOT NULL
                          AND account_name IS NOT NULL AND account_number_enc IS NOT NULL)),
  CONSTRAINT wallet_fields CHECK ((kind <> 'bank') = (wallet_e164 IS NOT NULL)),
  CONSTRAINT review_only_bank CHECK (status <> 'pending_review' OR kind = 'bank'),        -- only bank
     accounts need the name check
  CONSTRAINT usable_has_effective CHECK (status NOT IN ('cooling_off', 'active') OR effective_at IS NOT NULL)
     ,
  CONSTRAINT bank_reviewed CHECK (kind <> 'bank' OR status NOT IN ('cooling_off', 'active') OR reviewed_by IS
      NOT NULL)
);
-- one usable (or pending) payout account per seller
CREATE UNIQUE INDEX payout_account_one_current ON payments.payout_accounts (user_id)
  WHERE status IN ('pending_review', 'cooling_off', 'active');

CREATE TABLE payments.payouts (
  id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id           uuid NOT NULL UNIQUE REFERENCES orders.orders(id),    -- one payout per order
  seller_id          uuid NOT NULL REFERENCES identity.users(id),
  payout_account_id  uuid REFERENCES payments.payout_accounts(id),
  amount             bigint NOT NULL CHECK (amount > 0),
  status             payments.payout_status NOT NULL,
  hold_reason        varchar(120),
  simulated          boolean NOT NULL,                       -- true in the academic deliverable (FR-PAY-19)
  idempotency_key    uuid NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  provider_ref       varchar(120),
  attempts_made      smallint NOT NULL DEFAULT 0,
```

```
  next_attempt_at    timestamptz NOT NULL DEFAULT now(),
  created_at         timestamptz NOT NULL DEFAULT now(),
  sent_at            timestamptz,
  CONSTRAINT payout_sent_time CHECK ((status = 'sent') = (sent_at IS NOT NULL)),
  CONSTRAINT payout_needs_account CHECK (status IN ('waiting_account', 'held') OR payout_account_id IS NOT
     NULL),
  CONSTRAINT payout_hold_reason CHECK ((status = 'held') = (hold_reason IS NOT NULL))
);
CREATE INDEX payouts_due ON payments.payouts (next_attempt_at) WHERE status IN ('pending', 'failed', '
    waiting_cooling_off');

-- ====================================================================
-- Double-entry ledger (FR-PAY-12/13/14, FR-MON-09, DR-05)
-- Debits are positive, credits negative; every entry sums to zero.
-- ====================================================================
CREATE TABLE payments.ledger_accounts (
  code         varchar(40) PRIMARY KEY,
  kind         varchar(10) NOT NULL CHECK (kind IN ('asset', 'liability', 'revenue', 'expense')),
  description  text NOT NULL
);
INSERT INTO payments.ledger_accounts (code, kind, description) VALUES
  ('provider_clearing',       'asset',     'Funds␣held␣at␣payment␣providers␣(per␣provider␣in␣line.party)'),
  ('payout_clearing',         'asset',     'Funds␣sent␣out␣to␣sellers␣and␣couriers,␣awaiting␣provider␣
     settlement'),
  ('seller_receivable',       'asset',     'Amounts␣owed␣back␣by␣sellers␣(chargebacks)'),
  ('escrow_liability',        'liability', 'Buyer␣money␣held␣in␣escrow,␣per␣order'),
  ('seller_payable',          'liability', 'Released␣money␣owed␣to␣sellers'),
  ('courier_payable',         'liability', 'Delivery␣fees␣owed␣to␣couriers'),
  ('buyer_refund_payable',    'liability', 'Refunds␣owed␣to␣buyers'),
  ('revenue_commission',      'revenue',   'Transaction␣commission␣(FR-MON-01)'),
  ('revenue_delivery_margin', 'revenue',   'Delivery␣margin␣(FR-MON-03)'),
  ('revenue_featured',        'revenue',   'Featured␣listing␣fees␣(FR-MON-05)'),
  ('revenue_subscription',    'revenue',   'Pro␣Seller␣subscriptions␣(FR-MON-06)'),
  ('expense_courier_returns', 'expense',   'Courier␣charges␣for␣failed␣deliveries␣(FR-DLV-12)');

CREATE TABLE payments.journal_entries (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  kind        payments.entry_kind NOT NULL,
  order_id    uuid REFERENCES orders.orders(id),
  source_id   uuid,                        -- attempt, refund, payout ...
  memo        text,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX journal_order ON payments.journal_entries (order_id);

CREATE TABLE payments.journal_lines (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  entry_id      bigint NOT NULL REFERENCES payments.journal_entries(id),
  account_code  varchar(40) NOT NULL REFERENCES payments.ledger_accounts(code),
  party_id      uuid,                      -- user, provider-specific clearing, courier ...
  amount        bigint NOT NULL CHECK (amount <> 0)
);
CREATE INDEX journal_lines_entry ON payments.journal_lines (entry_id);
CREATE INDEX journal_lines_account ON payments.journal_lines (account_code, party_id);

-- every journal entry must balance; checked at commit, so lines can be inserted one by one
CREATE FUNCTION payments.check_entry_balanced() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE s bigint; n int;
BEGIN
  SELECT coalesce(sum(amount), 0), count(*) INTO s, n FROM payments.journal_lines WHERE entry_id = NEW.
     entry_id;
  IF n < 2 OR s <> 0 THEN
    RAISE EXCEPTION 'journal␣entry␣%␣does␣not␣balance␣(lines␣%,␣sum␣%)', NEW.entry_id, n, s
      USING ERRCODE = 'check_violation';
  END IF;
  RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER journal_balanced AFTER INSERT ON payments.journal_lines
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION payments.check_entry_balanced();
CREATE TRIGGER journal_entries_append_only BEFORE UPDATE OR DELETE ON payments.journal_entries
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();
CREATE TRIGGER journal_lines_append_only BEFORE UPDATE OR DELETE ON payments.journal_lines
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- ====================================================================
```

```
-- Settlement record per order (FR-PAY-13/14) and the fee calculation
-- ====================================================================
CREATE TABLE payments.settlements (
  order_id          uuid PRIMARY KEY REFERENCES orders.orders(id),
  amount_collected  bigint NOT NULL CHECK (amount_collected > 0),
  item_price        bigint NOT NULL CHECK (item_price > 0),
  delivery_fee      bigint NOT NULL CHECK (delivery_fee >= 0),
  commission        bigint NOT NULL CHECK (commission >= 0),
  courier_payout    bigint NOT NULL CHECK (courier_payout >= 0),
  delivery_margin   bigint NOT NULL CHECK (delivery_margin >= 0),
  seller_payout     bigint NOT NULL CHECK (seller_payout >= 0),
  refund_total      bigint NOT NULL CHECK (refund_total >= 0),
  commission_bps    integer NOT NULL,
  delivery_margin_bps integer NOT NULL,
  created_at        timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT settlement_collected CHECK (amount_collected = item_price + delivery_fee),
  CONSTRAINT settlement_parts_sum CHECK (commission + courier_payout + delivery_margin + seller_payout +
     refund_total = amount_collected),
  CONSTRAINT settlement_fee_parts CHECK (courier_payout + delivery_margin <= delivery_fee)
);
CREATE TRIGGER settlements_append_only BEFORE UPDATE OR DELETE ON payments.settlements
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- Reference implementation of the split (FR-PAY-11, FR-PAY-14, FR-MON-01, FR-DLV-13).
-- The application implements the same rules; tests compare the two.
--   item_refund : part of the item price refunded to the buyer (0 .. item_price)
--   courier_done: true if the courier picked the item up. If it did, the delivery fee is earned
--                 (courier payout + margin); if not, the whole fee goes back to the buyer.
-- The parts therefore always add up to item_price + delivery_fee.
CREATE FUNCTION payments.split(item_price bigint, delivery_fee bigint, item_refund bigint, courier_done
     boolean,
                               commission_bps int, margin_bps int)
RETURNS TABLE (commission bigint, courier_payout bigint, delivery_margin bigint,
               seller_payout bigint, refund_total bigint)
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
  released bigint;
BEGIN
  IF item_refund < 0 OR item_refund > item_price THEN
    RAISE EXCEPTION 'item␣refund␣%␣out␣of␣range␣0..%', item_refund, item_price;
  END IF;
  released   := item_price - item_refund;                            -- refund comes from the item price
     first
  commission := (released * commission_bps + 5000) / 10000;          -- half-up to the poisha
  IF courier_done THEN
    delivery_margin := (delivery_fee * margin_bps + 5000) / 10000;
    courier_payout  := delivery_fee - delivery_margin;
    refund_total    := item_refund;
  ELSE
    delivery_margin := 0;
    courier_payout  := 0;
    refund_total    := item_refund + delivery_fee;
  END IF;
  seller_payout := released - commission;                             -- remainder, calculated last
  RETURN NEXT;
END $$;

-- ====================================================================
-- Chargebacks (FR-PAY-20, Could), reconciliation issues (FR-PAY-15)
-- ====================================================================
CREATE TABLE payments.chargebacks (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  attempt_id        uuid NOT NULL REFERENCES payments.attempts(id),
  provider_case_id  varchar(120) NOT NULL UNIQUE,
  amount            bigint NOT NULL CHECK (amount > 0),
  status            varchar(20) NOT NULL CHECK (status IN ('open', 'lost', 'won', 'recovered')),
  recovered_amount  bigint NOT NULL DEFAULT 0 CHECK (recovered_amount >= 0),
  created_at        timestamptz NOT NULL DEFAULT now(),
  CHECK (recovered_amount <= amount)
);

CREATE TABLE payments.reconciliation_issues (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  kind         varchar(40) NOT NULL CHECK (kind IN
               ('amount_mismatch', 'late_payment', 'duplicate_payment', 'refund_failed', 'payout_failed',
```

```
               'provider_disagrees', 'stalled_delivery', 'manual_refund_required')),
  order_id     uuid REFERENCES orders.orders(id),
  attempt_id   uuid REFERENCES payments.attempts(id),
  refund_id    uuid REFERENCES payments.refunds(id),
  payout_id    uuid REFERENCES payments.payouts(id),
  detail       jsonb NOT NULL,
  status       varchar(10) NOT NULL DEFAULT 'open' CHECK (status IN ('open', 'resolved')),
  created_at   timestamptz NOT NULL DEFAULT now(),
  resolved_by  uuid REFERENCES identity.staff_accounts(id),
  resolved_at  timestamptz,
  resolution   text,
  CHECK ((status = 'resolved') = (resolved_by IS NOT NULL AND resolved_at IS NOT NULL AND resolution IS NOT
     NULL))
);
CREATE INDEX recon_open ON payments.reconciliation_issues (created_at) WHERE status = 'open';

-- ====================================================================
-- Monetization: featured listings and Pro Seller subscriptions (Could)
-- ====================================================================
CREATE TABLE payments.featured_purchases (                                    -- FR-MON-05
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id   uuid NOT NULL,                        -- Listings unit (no FK)
  seller_id    uuid NOT NULL REFERENCES identity.users(id),
  hours        smallint NOT NULL CHECK (hours IN (24, 48)),               -- CP-37
  fee          bigint NOT NULL CHECK (fee > 0),
  attempt_id   uuid REFERENCES payments.attempts(id),
  starts_at    timestamptz,
  ends_at      timestamptz,
  created_at   timestamptz NOT NULL DEFAULT now(),
  CHECK ((starts_at IS NULL) = (ends_at IS NULL)),
  CHECK (ends_at IS NULL OR ends_at = starts_at + make_interval(hours => hours))
);

CREATE TABLE payments.subscriptions (                                          -- FR-MON-06..08
  id                   uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id              uuid NOT NULL REFERENCES identity.users(id),
  status               varchar(12) NOT NULL CHECK (status IN ('active', 'past_due', 'cancelled', 'lapsed')),
  current_period_end   timestamptz NOT NULL,
  provider             payments.provider NOT NULL,
  provider_subscription_id varchar(120),          -- Stripe only; MFS renewals are paid each month
  failed_renewals      smallint NOT NULL DEFAULT 0,
  created_at           timestamptz NOT NULL DEFAULT now(),
  cancelled_at         timestamptz
);
CREATE UNIQUE INDEX subscription_one_live ON payments.subscriptions (user_id) WHERE status IN ('active', '
    past_due');

-- revenue log (FR-MON-09): every line on a revenue account, separate from settlements
CREATE VIEW payments.revenue_log AS
  SELECT e.id AS entry_id, e.created_at, e.kind, e.order_id, l.account_code, -l.amount AS revenue
    FROM payments.journal_entries e JOIN payments.journal_lines l ON l.entry_id = e.id
   WHERE l.account_code LIKE 'revenue\_%';

-- ====================================================================
-- Grants (SEC-INF-03). The application role cannot change schema or rewrite history.
-- ====================================================================
GRANT USAGE ON SCHEMA orders, payments TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA orders, payments TO tb_app;
REVOKE UPDATE ON orders.escrow_history, payments.journal_entries, payments.journal_lines,
                 payments.settlements, payments.ledger_accounts FROM tb_app;
GRANT SELECT ON payments.revenue_log TO tb_app;
GRANT EXECUTE ON FUNCTION payments.split(bigint, bigint, bigint, boolean, int, int) TO tb_app;
```